ChatGPT

# Parallelism for Training & Inference

## Introduction

Deep learning models have grown exponentially in recent years – for example, NLP models ballooned from the 340 million parameters of BERT-Large to 500 billion parameters in Megatron-Turing NLG within about four years. This surge in model size brings great accuracy gains, but it also makes training and inference extremely resource-intensive. Single GPUs often **cannot fit** these massive models in memory, and even if they could, **training them on a single device would be impossibly slow**. This is why **parallelism** techniques are critical: they allow us to spread the workload and memory footprint across multiple GPUs (and even multiple machines), enabling large models to be trained and served efficiently [1]. In this guide, we'll explore parallelism strategies for both training and inference of large Transformer models, including data, model (tensor and pipeline) parallelism, sharding (ZeRO/FSDP), activation checkpointing, CPU/NVMe offloading, and specialized inference optimizations like vLLM's paging and speculative decoding. Throughout, we will ground the discussion in practical examples – using a minimal GPT implementation (like nanoGPT) – and provide an annotated "lab" exercise demonstrating these concepts in code.

**Why Parallelism?** As model sizes increase, they quickly exceed the memory of a single accelerator and the compute needed grows beyond what one device can handle in a reasonable time. For training, the goal is often to **reduce training time** by using many GPUs in parallel (ideally with near-linear speedup) and to **fit large models** by dividing their parameters or states across devices. For inference, parallelism helps to **serve models that don't fit in one GPU's memory** and to **increase throughput or reduce latency** for generating predictions. All these come with trade-offs in complexity and communication overhead.

In the sections below, we break down the major forms of parallelism and memory optimization used in large-scale model training (data, tensor, pipeline parallelism, sharding, etc.), and then discuss how inference serving introduces its own parallelism challenges (multi-GPU serving, model paging, speculative decoding, etc.). Finally, we present a hands-on lab using **nanoGPT** as an example, showing how to modify a mini Transformer for distributed training with DeepSpeed ZeRO and how to serve it with vLLM, measuring performance.

---

## Parallelism in Training

Training large models efficiently requires distributing the work across multiple GPUs (and often multiple nodes). There are several fundamental **parallelism strategies** for training, each addressing different bottlenecks:

- **Data Parallelism (DP)** – Split the *data* across GPUs, each GPU has a full copy of the model.
- **Model Parallelism** – Split the *model* across GPUs (subdivided into **Tensor (intra-layer) parallelism** and **Pipeline (inter-layer) parallelism**).

- **Hybrid Parallelism** – Combinations of the above (e.g. using both data and model parallelism together, sometimes with pipeline parallelism as well – so-called 2D or 3D parallelism).
- **Memory Optimization Techniques** – Orthogonal strategies that don't change the algorithm's outcome but reduce memory usage: **ZeRO/sharding**, **activation checkpointing**, and **CPU/NVMe offloading**. These often pair with the above parallelisms to enable training of ultra-large models.

Let's examine each in detail.

## Data Parallelism

**Data parallelism** is the simplest and most widely used form of parallel training. In data parallelism, **each GPU (or process) gets a different slice of the input data** (e.g. different mini-batch samples), but **each GPU has a complete copy of the model parameters**. Each GPU runs the forward and backward pass on its data independently, producing gradients for its copy of the model. After the backward pass, the gradients are **synchronized (typically by all-reduce)** across GPUs to make sure all model copies stay in sync. Then each GPU updates its model parameters (usually all GPUs perform the same update, resulting in identical models). In summary: *"replicate the model, distribute the data."*

- **Implementation:** Modern frameworks make this easy. For example, PyTorch's `DistributedDataParallel` (DDP) handles gradient averaging under the hood. You simply ensure each process gets a different data shard each step.
- **Scaling:** If you have *N* GPUs, data parallelism ideally allows you to use an *N* times larger effective batch size (each GPU handles batch_size/N samples). This can drastically speed up training because *N* GPUs process *N* samples concurrently. In many cases, data parallelism yields the fastest training speed when the model is small enough to fit on one GPU, effectively giving almost linear speedups with more GPUs. It's often likened to "training with a higher batch size" – the wall-clock time per epoch decreases almost proportionally to the number of GPUs (minus some overhead) [2] .
- **Communication Cost:** During backprop, each GPU has computed gradients for its subset of data; those gradients must be averaged (or summed) across GPUs. This involves sending gradient tensors (equal in size to the model) between GPUs each step. For large models, this communication can be heavy, but it can often be overlapped with computation (for example, all-reducing gradients for earlier layers while later layers are still computing backward pass). Still, as you scale to many GPUs, the all-reduce can become a bottleneck if not managed well.
- **When to use:** Data parallelism is the go-to approach if your model **fits in one GPU's memory** and you want to train faster using multiple GPUs. It's also straightforward to use – you generally don't have to change your model implementation at all. However, if the model itself is too large for one GPU, pure data parallelism won't work (you can't even instantiate the model on a single GPU). That's when model parallel approaches become necessary. Additionally, extremely large batch sizes (from massive data parallel scaling) can lead to lower model quality or require adjusting hyperparameters (learning rate scaling, etc.), so there are diminishing returns after a point.

*Diagram conceptually*: Imagine 2-GPU data parallel training – we have model copy A on GPU0 processing mini-batch part1, and model copy B on GPU1 processing mini-batch part2. After backward pass, they exchange gradients and average them, so they both apply the same weight updates. Next iteration, they get new data and repeat. The model size stays the same on each GPU, but data is split.

## Model Parallelism

**Model parallelism** refers to techniques that partition the model's *layers or computations* across multiple GPUs, rather than replicating the whole model on each device. This is essential when the model is so large that it cannot fit into a single GPU's memory (for example, a 175B parameter model exceeds 40 GB memory in half precision). There are two primary flavors of model parallelism:

- **Tensor Parallelism (Intra-layer model parallelism):** Here, *individual layers* are split across GPUs. This is a **horizontal split** of the layer's parameters (hence "tensor parallel" – splitting the weight tensors). Each GPU holds a **partition of the neurons/parameters** in that layer and computes its part of the layer's output. At some synchronization points, the partial results from all GPUs are combined to form the full output. This approach was popularized by NVIDIA's **Megatron-LM** work, which showed that a Transformer's large fully-connected layers can be divided across GPUs with minimal changes to code (just insert a few all-reduce operations) [3].

- *Example:* Suppose a Transformer feed-forward layer has weight matrix $W$ of shape [d_model, 4*d_model] (as in GPT-like architectures where the hidden dimension is 4× the input dimension). In a 2-way tensor parallel setup, we split this weight into $W_1$ (first half of columns) and $W_2$ (second half) – each is [d_model, 2*d_model]. GPU0 holds $W_1$ and GPU1 holds $W_2$. In the forward pass, each GPU multiplies the input $X$ (shape [batch, d_model]) by its chunk of the weight: GPU0 computes $Y_1 = X \times W_1$, GPU1 computes $Y_2 = X \times W_2$. Now, to get the final output $Y = [Y_1 || Y_2]$ (concatenate on the hidden dimension), we need to exchange data between GPUs. In practice, one could all-gather the results so that each GPU has the complete $Y$, or one GPU could receive $Y_1$ from the other, etc. Similarly, in the backward pass, gradients w.r.t. $Y$ would be split and sent to the respective GPU to compute grad w.r.t $W_1$ and $W_2$. Another common pattern is splitting by output features and doing an all-reduce: e.g., each GPU produces an output of size [batch, d_model] (instead of half), then those outputs are summed (all-reduced) to form the final output. The exact partitioning strategy can vary by layer (for instance, one can split along the feed-forward intermediate dimension or split the attention heads across GPUs, etc.). The key is that each GPU does a portion of the layer's compute in parallel.

- *Communication:* Tensor parallelism typically requires **1 or 2 all-reduce operations per layer** (or matching all-gathers), which is relatively modest. Megatron-LM specifically noted their method only needed "a single all-reduce in the forward pass and one in backward" for their MLP layers [3], plus a couple for the attention mechanism. Because these communication operations happen at known points (e.g., after computing partial results), they can often be overlapped with other computations. And since each all-reduce is only within a group of GPUs that share the layer (e.g., groups of size 2, 4, or whatever the tensor parallel degree is), the communication scale is limited. In Megatron's case, they used 8 GPUs for tensor parallel, so each all-reduce was among 8 GPUs.
- *Benefits:* Intra-layer model parallelism lets you train **very large layers that would not fit on one GPU**. It also can improve speed for a single forward pass by using multiple GPUs for the same computation (if communication is not a bottleneck). Notably, Megatron-LM showed they could sustain high efficiency using this method – they trained an 8.3 billion parameter Transformer on 512 GPUs (with 8-way tensor parallelism × 64-way data parallelism) and achieved **76% scaling efficiency** relative to a strong single-GPU baseline. All communications were done via standard PyTorch operations (no custom C++ needed), proving that this kind of model parallelism can be implemented with a few lines of code changes.

- *Implementation:* Implementing tensor parallelism by hand requires modifying the model's forward pass to include communications. However, libraries exist to help: Megatron-LM (an NVIDIA library) provides distributed layers (like a distributed linear layer that automatically does the splits and all-reduce). PyTorch's Fully Sharded Data Parallel (FSDP) with "mixed precision for parameters" can mimic some tensor parallel behavior (sharding parameters) but typically one uses either Megatron-LM or DeepSpeed's integration for true intra-layer parallel. In our lab example later, we'll illustrate a simple manual approach for a 2-GPU tensor split.

- **Pipeline Parallelism (Inter-layer model parallelism):** In pipeline parallelism, we **split the model's layers into chunks (stages) and assign each stage to a different GPU**. For example, in a 4-GPU pipeline, each GPU might get 1/4 of the layers of the Transformer. The data (mini-batch) then needs to flow through the GPUs in sequence: first GPU0 processes the first few layers on the batch, then passes the intermediate activations to GPU1 which does the next layers, and so on. On the backward pass, gradients flow in reverse order. Pipeline parallelism by itself doesn't speed up processing of a *single* sample (since that sample still has to go through all layers sequentially), but it allows us to work on **multiple micro-batches in a staggered fashion** to keep all GPUs busy. This is known as the **"fill-drain"** pipeline scheduling (used by Google's **GPipe** framework).

- *Micro-batching:* If you only feed one batch through the pipeline at a time, many GPUs would be sitting idle except when their turn comes. For instance, GPU2 can't do anything until GPU1 hands it activations, which in turn waits for GPU0. This would be inefficient. Instead, you can split your mini-batch into smaller micro-batches and start feeding them in a stream. For example, micro-batch1 goes into GPU0, and as soon as GPU0 finishes layer1 on it and passes to GPU1, GPU0 can start processing layer1 on micro-batch2. This way, all GPUs can work in parallel on different micro-batches of the same overall batch. The first few micro-batches incur pipeline "bubbles" (initial fill where some GPUs are idle), and similarly at the end (drain), but if you have enough micro-batches, the utilization becomes high. GPipe (2018) demonstrated this can achieve nearly linear speedups if the batch is sufficiently large to be split.

- *Pipeline bubbles and overhead:* Despite micro-batching, pipeline parallelism often has some idle time per iteration, especially if the number of micro-batches is not much larger than the number of pipeline stages. There is also the issue of **load balancing** – each pipeline stage should ideally have equal amounts of work (compute time). If one stage (say the last layers) takes longer, it will become a bottleneck and other GPUs will idle waiting for it. Techniques like balancing the number of layers per stage or even **pipeline parallel with model parallel inside** (to equalize compute) can mitigate this. Nonetheless, pipeline parallelism introduces complexity: careful scheduling is needed to avoid incorrect weight updates (GPipe accumulates gradients and applies them only after all micro-batches complete the pipeline to maintain consistency).
- *Communication:* Pipeline parallelism's communication is shipping activations (and their gradients) between GPUs for each micro-batch. This is typically a point-to-point communication between stage i and stage i+1. The volume of data is the size of the activation tensor (for Transformers, that's batch_size × seq_length × hidden_dim for each transfer). Compared to gradient all-reduce in data parallel, this might be smaller or larger depending on batch and model dimensions. GPipe uses synchronous pipeline (waits for all micro-batches to finish a full forward+backward before updating weights) to avoid weight inconsistency issues.
- *Scaling:* Pipeline parallelism lets you **chain together many GPUs to fit very large models** (e.g., 3D parallelism uses pipelines of pipelines on multiple nodes). But it doesn't help with the *per step*

throughput as much as data/tensor parallel do, except that you can process a big batch in a pipelined fashion. Essentially, pipeline parallelism trades increased **throughput** (in terms of tokens or samples processed per time, if using micro-batches) for a bit more **latency** (one sample may see additional waiting time). As model sizes exceeded what even multi-GPU tensor parallel could handle, pipeline parallel became a necessary complement. For example, **Microsoft's Turing-NLG 17B** and larger models used pipeline+tensor parallel (as do many BigScience or MT-NLG runs).

- *Example:* If we have 2 GPUs and a model with 24 layers, we could put layers 1–12 on GPU0 and 13–24 on GPU1. Micro-batch A goes through GPU0's layers then GPU1's. While GPU1 is working on micro-batch A, GPU0 can work on micro-batch B's first 12 layers. By the time GPU0 is done with B, GPU1 is likely free to take B (assuming equal compute), and GPU0 can take micro-batch C, and so on. The pipeline is full after a couple of micro-batches. There will be one micro-batch's worth of bubble at the start and end. The more micro-batches (if the overall batch is split into, say, 8 micro-batches for 2 GPUs), the smaller the fraction of bubble time. Researchers have observed that pipeline bubbles can account for ~40% overhead if using too few micro-batches, but this drops as micro-batch count increases.
- *Pipeline vs. Tensor parallel:* Pipeline parallel is sometimes called **"vertical model parallelism"** (splitting the stack of layers), whereas tensor parallel is **"horizontal model parallelism"** (splitting within a layer). They can be combined: e.g., 4-way pipeline and 2-way tensor means your model is effectively on 8 GPUs (2 GPUs per stage doing each layer in parallel). This 8-GPU setup can then be further replicated for data parallel. That kind of combination (data × pipeline × tensor) is often termed "3D parallelism." For instance, the **8.3B Megatron-LM model** mentioned earlier was trained on 512 GPUs as 8-way tensor parallel × 8-way pipeline × 8-way data parallel (actually they report 8 tensor * 64 data = 512, which implies pipeline was not used for that specific run, but pipeline is used in other cases). Pipeline parallel tends to be used more in very large-scale setups, as it's harder to get benefits in small-scale (with 2–4 GPUs, often tensor or just data parallel is enough unless memory dictates).

**Hybrid Parallelism:** In practice, large training jobs use a **mix of data and model parallelism**. Data parallelism by itself requires each GPU to have a full model copy, which becomes infeasible beyond a certain model size. Model parallelism by itself (without data parallel) means you are using multiple GPUs to train one sample at a time – that might keep GPUs busy but you don't increase throughput unless you also increase batch size. By combining them, you can both scale to large models and process more data. For example, if you have 16 GPUs and a model that barely fits on 4 GPUs, you could use 4 GPUs for the model (tensor/pipeline parallel) and then replicate that across 4 groups (data parallel groups) – achieving 4× more throughput than a single model-parallel group, while still fitting the model. The total GPUs = (model parallel size) × (data parallel size). All groups run in parallel on different data, and within a group, the GPUs cooperate to run the model. This is how many flagship models are trained. The Megatron-LM paper explicitly notes: *"the total number of required GPUs is the product of the number of model and data parallel groups"*. In their example, they used 8-way model parallel and 64-way data parallel = 512 GPUS for the 8.3B model. They also note that pure model parallel achieved ~77% scaling efficiency (8 GPUs vs 1), and with the addition of data parallel (which introduces extra gradient all-reduce across the 64 groups) the overall efficiency was ~74% relative to ideal linear scaling. This shows that the overhead of combining them was relatively small (a few percent loss), which is a good outcome.

To summarize model parallelism: it is necessary for *fitting* large models and can also contribute to speeding up training by leveraging multiple GPUs for a single instance. It usually requires more complex code (or using libraries like Megatron/DeepSpeed) to handle splitting and communication. The state-of-the-art large models often use tensor parallel for splitting within layers (for raw speed and to avoid single-layer memory

bottlenecks) and pipeline parallel for splitting layers across nodes (for overall memory scaling), plus data parallel for additional scale. This hierarchy might be called 3D parallelism or "Megatron-Deepspeed" recipe. The good news is frameworks hide some complexity: for example, **DeepSpeed** provides a means to combine data, tensor, and pipeline parallelism relatively transparently, and **PyTorch FSDP** provides sharding (discussed next) that can complement model parallel.

## Memory Optimization Techniques (ZeRO, Sharding, Checkpointing, Offloading)

In addition to splitting work across GPUs, large-scale training also demands strategies to **manage memory usage** efficiently. Two major memory consumers during training are **model states** (parameters, gradients, optimizer states) and **activations** (needed for backward pass). If each GPU naively holds a full copy of all these, memory scales poorly. Let's discuss how techniques like **ZeRO (Zero Redundancy Optimizer)** and **activation checkpointing** address this, and how offloading to CPU/NVMe can further extend capacity.

- **Activation Checkpointing (Rematerialization):** This technique trades computation for memory. Normally, during forward propagation, we store the output of every layer (the activation) so that during backpropagation we can compute gradients. Storing all those intermediate activations can consume a lot of GPU memory, especially for deep networks and large batch sizes. Activation checkpointing (introduced by Chen et al. 2016) means we selectively **do not store some intermediates**, and instead **recompute them** from scratch in the backward pass. For example, in a 24-layer network, you might only store the activations after every 3rd layer. During backward pass, when you need the activation of layer 17 (which wasn't stored), you recompute layers 1–17 forward (or some segment of the network) to get it, then proceed with backprop for that segment. This increases the computation (some forward ops are done twice), but saves memory. In practice, frameworks allow you to designate checkpointed layers: PyTorch's `torch.utils.checkpoint` will automagically drop the saved activations for a given layer or block and recompute on backward. Activation checkpointing can dramatically reduce memory (nearly linear with how many layers you checkpoint), enabling either larger batches or deeper models to train. The cost is roughly one extra forward pass worth of compute for the whole model if done aggressively (which is usually acceptable when memory is the limiter). This technique is **orthogonal** to parallelism: it can be used together with any of the parallel strategies discussed. In fact, the Megatron-LM paper mentions they needed activation checkpointing to fit very large models in GPU memory. It's a standard tool for training models beyond the naive memory limit.

- **ZeRO and Sharded Data Parallelism: Zero Redundancy Optimizer (ZeRO)** is a family of techniques developed in DeepSpeed to reduce memory footprint by **sharding the model states (parameters, gradients, optimizer stats) across GPUs**, instead of each GPU having redundant copies. In plain data parallel, if you have *N* GPUs, you have N copies of the model and optimizer states – which is quite wasteful in terms of memory. ZeRO aims to eliminate this redundancy, hence "zero redundancy." It was introduced in stages:

- **ZeRO Stage 1:** Shard the **optimizer states** (e.g., Adam's momentum and variance buffers). Instead of every GPU storing Adam's full mt and vt for all parameters, each GPU stores only a slice of those for different parameters. For example, with 4 GPUs, each GPU might keep optimizer states for 1/4 of the parameters. This immediately saves memory (optimizer states can be 2-3× the size of the model for Adam). However, each GPU still had the full model weights and gradients in Stage 1.

- **ZeRO Stage 2:** In addition to optimizer states, shard the **gradients** as well. After backprop, instead of all GPUs storing all gradients, they partition them. Conceptually, after an all-reduce you could throw away most of the gradients and only keep the slice relevant to that GPU's parameter shard. Practically, DeepSpeed achieves this by using **reduce-scatter** rather than all-reduce: gradients from different parts of the model get reduced to different GPUs. The result is each GPU ends up with the gradient for only the parameters it "owns." This means during the optimizer step, that GPU can update those parameters (because it has its gradients and optimizer state shards for them). Stage 2 still keeps a full copy of the **parameters** on each GPU for forward/backward computation.
- **ZeRO Stage 3:** Shard **parameters** as well. This is the full realization of ZeRO: no replica has the full model. At any given time, a layer's parameters are distributed across GPUs; they are gathered when needed for computation and then scattered back out. Stage 3 is effectively equivalent to **Fully Sharded Data Parallel (FSDP)** in PyTorch – the model's weights are partitioned, and before running a layer, the relevant shards are all-gathered so that that layer's computation can happen, then after computing gradients, they are reduce-scattered. With ZeRO-3, memory usage can scale roughly inversely with number of GPUs for the model states (plus overheads for temporary full gatherings). This allows training models N times larger on N GPUs, in theory, without needing model parallel code changes inside layers (it's "model parallelism without model code changes" in a sense, handled by the runtime). It's worth noting that sharding the parameters means increased communication (gathering parameters for every layer, every forward pass), so it can introduce runtime overhead, but DeepSpeed and others manage this well with overlapping and making it efficient in C++ kernels.

To give a sense of impact: the ZeRO paper (Rajbhandari et al. 2020) demonstrated training **models over 100B parameters** with 400 GPUs using ZeRO, achieving **15 PFLOPs of throughput** and enabling an **8× larger model** than the previous state-of-the-art could (at that time). They even achieved a *super-linear speedup* in some cases because ZeRO allowed a larger batch size, which meant better hardware utilization [4]. They specifically mention training a model with *over 100 billion params* with **super-linear speedup on 400 GPUs**, and that **ZeRO could scale to 1 Trillion parameters** given enough hardware, by eliminating memory redundancies. Importantly, ZeRO stage 3 allowed them to train a **13 billion parameter model without any model parallelism** – previously 13B (like Megatron 8B or T5-11B) would have required model parallel, but ZeRO-3 could handle it by sharding across, say, 8 GPUs transparently. This greatly simplifies life for researchers: you don't need to rewrite your model for model parallel if FSDP/ZeRO can just shard it for you. (Of course, beyond certain sizes, combining ZeRO with model parallel might be needed for performance or if a single layer is too large to even temporarily all-gather on one GPU.)

In practice, **Sharded DDP** (PyTorch's implementation of ZeRO-3 concept) and DeepSpeed ZeRO have become common for training large models. They reduce memory per GPU, which can be traded for either larger models or larger batch sizes (which can improve utilization). They do add communication overhead, but newer optimizations (like **ZeRO-2 Offload, ZeRO-Infinity, and ZeRO++ for comm efficiency**) alleviate many issues. The Hugging Face Accelerate library, for instance, can automatically turn on DeepSpeed ZeRO-3 or PyTorch FSDP for your model, enabling training of models like 20B, 30B on multi-GPU setups that otherwise wouldn't fit.

It's helpful to contrast **ZeRO vs Model Parallel**: ZeRO (sharded data parallel) slices model *states* at a high level, but at runtime each shard still ends up getting the data it needs (like gathering a layer's weights to do the forward). Pure model parallel (like tensor parallel) slices the actual compute within each layer so no GPU ever holds the full layer's weights at once during compute (they hold their partition only). ZeRO-3 does temporarily assemble full layers (in memory) during compute, so it's sort of a different approach (less efficient compute-wise since there's redundant calc across shards? Actually in ZeRO-3 there isn't redundant

calc, just extra communication). Interestingly, you can **combine ZeRO with model parallel**: e.g., if you have 8 GPUs for tensor parallel and 8 for data parallel, you can also shard optimizer states across those 8 data-parallel replicas (DeepSpeed supports this). DeepSpeed also has something called **Hybrid Engine** to integrate with Megatron-LM's tensor parallel so you get the best of both.

**ZeRO Stage 0** (for completeness) is basically regular Data Parallel (no sharding) – Accelerate refers to ZeRO-0 as just DDP. So Stage 1/2/3 progressively increase sharding.

- **Offloading to CPU/NVMe (ZeRO-Infinity):** Even with ZeRO and model parallel, some models are beyond reach of GPU memory. For instance, in 2022 the largest models like MT-NLG 530B or PaLM 540B – even split across many GPUs – are at the edge of capacity. Also not everyone has access to dozens of GPUs. To address this, techniques have been developed to use **CPU RAM or even disk (NVMe SSD)** as extended memory for training. **ZeRO-Infinity** (DeepSpeed) was one such innovation: it generalizes ZeRO by allowing each shard to reside in **hierarchical memory tiers** and swap between them as needed. Essentially, **keep most of the model/optimizer in CPU memory, and bring layers into GPU memory just-in-time** to compute. After computation, move them out if necessary to free space. This is like using the CPU as swap memory for GPU. NVMe (fast SSD) can further be used if CPU RAM isn't enough (since NVMe can be hundreds of GB, but slower). DeepSpeed reported that using ZeRO-Infinity, they could **train 2.5 trillion parameter models** in simulation by leveraging NVMe, and practically they trained multi-hundred-billion models on a single node with limited GPU memory by offloading most data to CPU/NVMe.

Offloading introduces a performance hit (PCIe transfers are much slower than VRAM). But if done intelligently (overlap communication with computation, and exploit that not all weights are needed at once), it can be workable. The ZeRO-Infinity approach is: partition model across GPU, CPU, NVMe such that the working set for the current operation fits in GPU. This may involve *prefetching* next layers into GPU while the current one is computing, and *evicting* unused ones. The question is: when is this worthwhile? If the compute on the layer is large (e.g., big matrix multiplications on a large batch), the GPU can be busy for, say, 10 milliseconds, during which time you might be able to transfer a few hundred MB from CPU to GPU (PCIe ~12 GB/s). For large batches or sequence lengths, each layer's compute time dominates, so the data transfer can be overlapped and become a minor part of the total time. In contrast, for very small batches, the compute per layer is so fast that moving data from CPU would stall the pipeline (this is why offload-based methods aim at throughput-oriented scenarios).

ZeRO-Infinity for training is complex but has enabled training beyond normal limits. For example, they mention that with offloading, **half-precision inference of a 530B model went from needing 1 TB of GPU memory to just 10 GB** (100× reduction) by streaming layers from CPU. For training, similarly, the *GPU* memory needed at any time can be a small fraction of the total model size.

We'll revisit offloading in the inference context (ZeRO-Inference) which is a related concept, but the main idea is: *use cheaper memory tiers (CPU RAM, disk) to hold model parameters or optimizer states that are not immediately needed, thereby reducing expensive GPU memory usage*. This **significantly reduces cost**, since CPU RAM and NVMe are far cheaper per GB than GPU VRAM, and it **increases accessibility** (one could, in principle, fine-tune a 100B model on a single GPU if willing to accept the slowdown from paging data in/out).

**Summary of Memory Techniques:** Activation checkpointing and ZeRO are often used together. For instance, in the BigScience 176B model training (BLOOM), they used activation checkpointing to keep memory manageable, and they used Megatron (tensor + pipeline) for model parallelism *and* ZeRO for optimizer+gradient sharding. All these pieces combined allowed training that massive model on a cluster of GPUs. The trade-offs to keep in mind:

- **Checkpointing**: + saves memory, – adds ~1.5× training time (due to recompute).
- **ZeRO (sharding)**: + saves a ton of memory, – adds communication overhead.
- **Offload**: + breaks memory limit entirely, – can slow down training drastically if not enough overlap (requires fast interconnects and large batch to be efficient).
- These typically do not change the convergence or final model quality (they are mostly system optimizations). Checkpointing can slightly change numerical results but not significantly.

## Performance and Cost Considerations in Training

When scaling training to multiple GPUs or nodes, one should consider the **efficiency** and the **cost** (in time or money):

- **Strong Scaling vs Weak Scaling:** *Strong scaling* refers to speeding up a fixed total problem size by adding more resources (e.g., train the same model on the same dataset faster with more GPUs). *Weak scaling* refers to increasing the problem size proportionally to resources such that runtime stays constant (e.g., train a bigger model or use a larger dataset when you have more GPUs). In deep learning, we usually care about strong scaling (can we finish training in 1 day instead of 4 days by using 4× GPUs?). Many parallelism techniques strive for strong scaling, but often there's a trade: to get good efficiency, we may end up effectively doing weak scaling (e.g., using larger batch sizes when we add GPUs – which keeps GPUs busy and even improves throughput super-linearly, but it changes the training dynamics slightly).

- **Scaling Efficiency:** This is typically measured as `Speedup / #GPUs`. If you use 8 GPUs and get 7× speedup over 1 GPU, that's ~87.5% efficiency. Some loss is expected due to overheads. In the Megatron-LM example, 512 GPUs achieved 76% efficiency vs 1 GPU – quite good at that scale. But note, their baseline single GPU was probably not utilizing as large a batch as 512 could (they mention the single GPU baseline was 1.2B model, whereas with 512 GPUs they did 8.3B model at batch that gave best utilization). So sometimes the "single GPU baseline" is not apples-to-apples if the large configuration also enabled a larger model or batch. It's common to see near-linear scaling up to some number of GPUs, after which efficiency drops off. For data parallel, one bottleneck is the all-reduce of gradients – as you increase GPUs, the amount of data each GPU processes per step might go down (if you don't increase batch size), so computation per GPU decreases linearly, but communication per step doesn't decrease as much (since it's proportional to model size, not batch). Eventually communication dominates and adding more GPUs gives diminishing returns. One mitigation is to increase batch size with GPUs (to keep each GPU busy with more work). This is often done until an algorithmic limit is reached (too large a batch hurts convergence).

- **Communication Overhead:** A useful mental model: if each GPU has P FLOPs of compute per step and needs to communicate C bytes per step (with bandwidth B bytes/sec), then adding GPUs reduces P per GPU (if batch per GPU is fixed or even if not, the total compute per GPU might not increase proportionally). The communication time per step might be ~C/B (assuming perfect overlap

or sequential depending). For scaling to remain efficient, we want P (the work per GPU) to remain high relative to C/B. Techniques like **overlap** can hide comm latency under compute. For example, overlapping gradient reduction with backward compute can hide much of the all-reduce time as long as backward compute > all-reduce latency. This is often achievable by chunking gradients and using non-blocking all-reduce.

- **Latency vs Throughput in Training:** Training is usually throughput-oriented (how many samples or tokens per second can we process). However, some parallelism (like pipeline) increases latency per step (because of micro-batch sequencing) even if throughput (per epoch speed) is good. This usually isn't a big concern unless there's a strict time dependency (like RL with environment interaction, where latency matters). For static datasets, throughput is king.

- **Cost in $ or energy:** Larger clusters obviously cost more to run. There is often an optimal point where adding more GPUs yields diminishing returns such that the **cost (GPU-hours) to train a model to a certain quality** starts increasing. For example, if 2 GPUs give 1.9× speedup (95% efficient), that's great and cost-effective (same total GPU-hours roughly). If 32 GPUs give only 16× speedup (50% efficient), you are spending 32 GPU-hours for work that 16 GPU-hours could do on fewer GPUs – not cost-optimal, though it is faster in wall-clock. In corporate settings, wall-clock time is valuable so they might use thousands of GPUs even at low efficiency. In academic or personal settings, one might prefer fewer GPUs running longer to save cost. This is a consideration when planning large runs.

- **Memory vs Compute Trade-offs:** Some strategies allow trading memory for compute (checkpointing) or memory for network IO (offloading). If you are memory-bound (model doesn't fit), you spend in either extra compute or comm to make it work. The "cost" of those should be justified by enabling something otherwise impossible (like fitting the model at all). An example cost analysis: if offloading each layer from CPU adds, say, 10% time overhead, but saves you from buying 4 more GPUs, that might be a worthwhile trade. If checkpointing adds 30% time but lets you train with a 2× larger batch (which might actually reduce steps needed for convergence in some regimes), it could be beneficial overall.

- **Empirical measurement:** It's good practice to measure how well your parallel setup is scaling. E.g., measure time per iteration on 1, 2, 4, 8 GPUs. You might find that beyond 4 GPUs, the gains flatten out because, say, your batch size per GPU got too small and you hit kernel launch overheads or communication dominated. Tools like NVIDIA's Nsight Systems or PyTorch's profiler can show timeline overlaps to see if all-reduces are stalling your compute.

In summary, to train large models: we partition the work (data, model, or both), we optimize memory usage (checkpointing, sharding, offload), and we try to maximize GPU utilization. Modern libraries like **DeepSpeed, Horovod, PyTorch DDP/FSDP, and Hugging Face Accelerate** provide these capabilities with high-level APIs. For instance, Accelerate will ask if you want to use DeepSpeed or FSDP and set up your training loop accordingly. The heavy lifting (gradient syncing, sharding, etc.) is mostly automated. Still, understanding these concepts is important to debug and to design efficient training schemes (like how to set micro-batch count for pipeline, or when to enable certain ZeRO features). The **bottom line** is that parallelism, done right, can unlock enormous models and speed up training by orders of magnitude – turning a months-long training on one GPU into a days-long training on a cluster.

---

# Parallelism in Inference

After training a large model, the next challenge is **serving** it: using the model to make predictions/generate text for users or downstream tasks. Inference has different characteristics than training. Training is a long-running, throughput-oriented job where latency of each batch is less important than overall samples/second. Inference (especially for interactive applications like chatbots) often cares about **latency** (how fast can we get the result for a single input) as well as **throughput** (how many queries can we handle per second). Also, during inference we don't update weights, so we don't need to store gradients or optimizer states – memory is mostly consumed by model weights and **activation buffers (especially the autoregressive *KV cache* in Transformers)**.

Key parallelism considerations for inference:

1. **Model too big for one GPU?** If yes, we need to distribute the model across multiple devices (similar to model parallel in training). If no (fits in one GPU), it might actually be simplest to serve from one GPU (to avoid multi-GPU communication latency) and scale out with data parallel copies if needed for throughput.
2. **Need to reduce latency for a single query?** Strategies include using multiple GPUs to handle one query faster, or algorithmic techniques like speculative decoding.
3. **Need to increase throughput (queries per second)?** Strategies include batching multiple queries together, using multiple replicas of the model (horizontal scaling), and efficient caching of computation.

We'll discuss the parallelism techniques specifically used in inference:

## Model Parallel Inference (Sharding the Model at Inference Time)

This is analogous to model parallel training. If a model is so large it cannot reside in one GPU's memory (even just for forward pass), or if we want to leverage multiple GPUs to speed up a single inference, we can **shard the model across GPUs**:

- **Tensor-parallel inference:** We partition the model's weights across *n* GPUs and perform each forward pass in parallel across those GPUs. For example, the same Megatron-LM style tensor parallel can be used just for inference: each GPU holds a slice of each layer and they jointly compute the forward. The communications (all-reduces/all-gathers) are the same as in training, except there's no backward. This can nearly linearly speed up the forward computation if implemented well, since each GPU does 1/n of the dot-product work (with some overhead for combining results). The requirement is that all GPUs must be available and synchronized for each inference request (they operate like a single pipeline).
- *When to use:* Tensor parallel inference is useful to deploy extremely large models (e.g., 175B GPT-3) in a way that each token's computation is distributed. If one GPU would take 100 ms to compute a forward, 4 GPUs tensor-parallel might do it in ~25 ms (plus some overhead). It's also needed if the model literally doesn't fit in one GPU memory – e.g., GPT-3 175B (requires > 80 GB just for weights in half precision) must be split across multiple GPUs to be hosted.
- *Example:* NVIDIA's **TRITON inference server** and **FasterTransformer** library support model sharding. You might load a model checkpoint that is split into 8 files (for 8 GPUs) and each GPU loads

its part. During inference, one GPU will handle, say, part of each matrix multiply and an all-reduce will sum up attention outputs, etc. The user just sees one logical model.

- *DeepSpeed's tensor slicing:* The DeepSpeed-Inference system (which we discuss in ZeRO-Inference below) can also do a form of tensor parallel on the fly. As per their documentation, if a layer is too large for one GPU, they can **split the layer across multiple GPUs** instead of loading it whole, avoiding that GPU's memory overflow. This is essentially the same concept: each GPU stores a chunk of the layer and they compute together.

- There is generally **no fundamental difference** between how you'd do tensor parallel in training vs inference, except that inference could potentially use larger batch sizes per forward (since no backward), which might make communication relatively less significant. Also, for autoregressive generation, you have to do many sequential forward passes (one per generated token), so any latency per token adds up. If tensor parallel can reduce that latency, it can directly speed up generation.

- **Pipeline-parallel inference:** This would involve splitting layers among GPUs (like stage 1 on GPU0, stage 2 on GPU1, etc.) and then feeding the input through them. However, pipeline parallel by itself *does not reduce the latency for a single input*, because the input still has to go through stage1 then stage2 sequentially. In fact, it might slightly increase latency due to device-to-device transfer overhead. The benefit of pipeline parallel in inference is mainly to **enable hosting a model that otherwise wouldn't fit** (by splitting it) or to allow **concurrent processing of multiple inputs** in a staggered way (to increase throughput).

- If you have multiple requests, you could form a pipeline: while one request's data is in later layers, a new request's data can start in the earlier layers – thus all GPUs stay busy. This is similar to training's micro-batching concept, applied to inference requests. Frameworks like **PyTorch Pipeline Parallel** or **Microsoft ORT** can do this. But careful: In generation tasks, each new token depends on the previous token's output, so you can't pipeline different time steps of the *same sequence*. You can pipeline different sequences. For instance, if you have 2 pipeline stages, you could handle two separate sequences in a ping-pong fashion: sequence A flows through stage1→stage2, and as soon as A leaves stage1, sequence B enters stage1 while A is in stage2. This way, you achieve throughput 2x without adding devices beyond what's needed to hold the model. This is effectively just batching two sequences, but pipeline parallel ensures each part of the model is only on one GPU, so it's necessary when the model is split.

- Usually, if a model fits on one GPU, you'd prefer to keep it there for inference because splitting it (pipeline) would just introduce overhead. Pipeline parallel inference thus is primarily for model-size reasons: e.g., if you have a 60B model and only 2 32GB GPUs, you might put half the layers on each. The latency might go up a bit vs if you had a hypothetical 64GB GPU, but at least you can run the model at all. One slight latency gain you can get is if the two stages can work on *different* queries in parallel as described, but that's more about throughput.

- **Parameter Offloading at Inference:** Another approach if the model is huge for one GPU is not to use multiple GPUs at all, but to offload weights to CPU and stream them in as needed. This is what **ZeRO-Inference** (DeepSpeed) does, which we'll cover soon. It's akin to model parallel across time rather than space: instead of spreading the model across GPUs simultaneously, you load part now and part later. This doesn't reduce per-token latency (in fact it can increase it), but it reduces GPU

memory requirement to basically zero (hence democratizing usage on a single GPU with large CPU memory). We mention it here as an alternative to multi-GPU model parallel for serving large models.

- **"TP-Sharding" (Tensor Parallel + Sharding):** This term in the prompt likely refers to using **both** model parallel and sharding concurrently for inference. For example, you might split each layer across 2 GPUs (tensor parallel), *and* replicate that setup for 2 more GPUs with weights sharded between them? However, typically "TP + sharding" in inference could mean something like *Each pipeline stage is model-parallel across some GPUs, and you shard some layers...* It's a bit ambiguous. One clear interpretation from the context is the DeepSpeed inference discussion: *"does ZeRO-Inference integrate with model parallel tensor-slicing? Yes it does"*. So you can have, say, ZeRO-Inference offloading + *also* do tensor parallel among GPUs to split layers that are too big. This is a very advanced scenario: e.g., imagine a 1 trillion parameter model – you might need both CPU offload and multi-GPU to host it. In simpler terms, think of **TP-sharding** as *combining distributed data parallel and model parallel for inference*, similar to training. For instance, if you have 4 GPUs, you could use 2-way tensor parallel on 2 GPUs for one instance of the model and another 2 GPUs for a second instance (data parallel replication). This would allow serving two queries simultaneously (one on each model replica which itself is 2 GPUs working together). This improves throughput (two queries at once) while each query benefits from model parallel speedup. This pattern might be used in high-throughput servers: they launch multiple model replicas across the cluster, each model replica itself is multi-GPU.

- **Data parallel inference:** This is straightforward: if you want to handle more queries per second and your model fits on one GPU, you can run multiple copies of the model on multiple GPUs. Each copy handles different requests independently. This is essentially horizontal scaling of a stateless service. Many production deployments do this – e.g., if one GPU can serve X QPS, 4 GPUs can serve ~4X QPS (assuming the load is evenly distributed). There's usually a load-balancer or scheduler sending requests to different GPU workers. Data parallel inference does **not** reduce the latency of a single query (each query still goes to one model instance), but it increases aggregate throughput. It's most useful when you have lots of independent requests (e.g., many users' queries to an API). If you have few but very large requests (like one giant batch), then you might prefer model parallel to reduce latency for that batch.

- In frameworks like TensorFlow Serving or ONNX Runtime, this is done by just launching multiple model sessions. In PyTorch, one might spawn multiple processes (each pinned to a GPU) running the model server logic.
- There's little special to discuss here except that it's embarrassingly parallel – no communication between GPUs needed since each is working on different data. The challenge comes in utilization: if request rate is low, those GPUs might be idle a lot. Some solutions (like vLLM, next section) aim to **dynamically batch requests together on fewer GPUs** to maximize utilization, rather than statically allocating one GPU per request thread.

**Memory and the KV Cache:** A distinctive aspect of Transformer inference, especially for generative LLMs, is the **key-value cache**. When generating text with a Transformer, after processing the initial prompt, the model caches the key and value vectors from each attention layer for the prompt tokens. As new tokens are generated one by one, the cached K/V from previous tokens are reused so that we don't recompute the entire attention from scratch. This speeds up generation massively for long sequences because each new token's computation only does attention over previous tokens via the cache instead of recomputing all those earlier layers repeatedly.

However, the KV cache is large: for a model like GPT-3, each new token adds (layers × hidden_dim × 2) floats to the cache (times batch size). For long outputs (say 1000 tokens generated) this becomes huge. For example, vLLM authors noted *"the KV cache takes up to 1.7 GB for a single sequence in LLaMA-13B"* [5] (that's with full 2048 token context). Additionally, if you serve many sequences of varying lengths, managing this memory becomes tricky – a naive approach might allocate the maximum possible size (for worst-case length) for each sequence, leading to a lot of wasted space when sequences are shorter.

We bring this up because **parallelism and memory management for inference** is not only about model weights, but also about handling this cache efficiently. A conventional HF Transformers server might pre-allocate a huge tensor for cache (e.g., [batch_size, num_layers, 2, num_heads, seq_len, head_dim]) and slice it per request. This can waste memory if actual seq_len per request is smaller, etc. And it can limit how many concurrent sequences you serve (because you allocate worst-case memory for all).

We'll now discuss specialized solutions addressing these inference-time challenges:

## Memory Management and Optimized Batching in Inference: vLLM and PagedAttention

One of the recent breakthroughs in serving LLMs is the **vLLM** library (2023), which introduced a new attention backing store called **PagedAttention**. This directly tackles the KV cache issue mentioned above.

**vLLM's PagedAttention** [6] : The idea is inspired by virtual memory in operating systems. Instead of one giant contiguous allocation for the attention keys/values of each sequence, vLLM **breaks the KV cache into small blocks ("pages")** – for example, blocks that can hold, say, 16 or 32 tokens worth of key/value. These blocks are then allocated on demand and can be **non-contiguous in physical memory**. A mapping (like a page table) keeps track of which pages belong to which sequence and in what order. So, as a sequence grows, vLLM will allocate new blocks for its new tokens. If sequences end or aren't using some allocated capacity, that memory can be reused for others. This largely eliminates the fragmentation and over-allocation problem: **memory is allocated only as needed** and almost all of it holds useful data (little padding) [6] . In their results, they found this scheme reduced memory waste from ~60–80% (in traditional setups) to under 4% [5] . Essentially, vLLM achieves *near optimal GPU memory utilization* for the KV cache.

**Continuous batching and throughput:** With flexible memory, vLLM can do something clever – **continuous batching of requests**. Normally, servers batch requests that arrive at the same time. But if a slow stream of requests comes, you either wait to batch them (incurring latency) or handle them one by one (incurring under-utilization). vLLM's approach allows it to batch even asynchronous streams effectively. Because it can intermix different sequences' tokens, it will always try to fill the GPU with as much work as possible at each step. For example, suppose one user request has generated 10 tokens and is waiting for the 11th, and a new user request comes in (starting token 1). Rather than serving them sequentially, vLLM can *combine* these two in one forward pass by treating them as a batch of two sequences (one with past length 10, one with past length 0). It manages the different sequence lengths easily because of the paging (no need for large padding tensors). This is tricky in naive implementations because different past sequence lengths require masking and often lead to padding to max length. vLLM handles it by only bringing in the necessary "pages" for each sequence in the attention computation [7] . The result is **higher throughput**: the GPU is kept busy with multiple sequences at once. In experiments, vLLM achieved order-of-magnitude higher throughput than naive implementations. For example, serving LLaMA-13B, it was **14× to 24× faster throughput than HuggingFace Transformers** (without TGI) and about **3× faster than HuggingFace's optimized TGI server**. The gains are especially pronounced when each request is relatively small (so

without batching you'd waste time). Even when each request asks for multiple parallel completions (like generating 3 samples for one prompt), vLLM can share the prompt memory across those and still outperform others [8].

**Memory sharing:** vLLM also implements a form of *copy-on-write for the cache*. If you have scenarios like *parallel decoding* (one prompt, generating N different continuations in parallel, as used in beam search or producing multiple samples), a normal implementation would duplicate the prompt's key/value tensors N times for each continuation. vLLM, however, can let those N sequences **share the same physical memory for the identical prefix**, only diverging when new tokens are added. This is analogous to processes in an OS forking and sharing memory pages until a write occurs. They maintain reference counts for pages and duplicate only when needed. This **reduces memory usage in multi-sample inference by up to 55%**, and because less memory is used (and fewer transfers), it also improved throughput by up to 2.2× in those scenarios. Complex decoding algorithms like beam search become much more feasible with this optimization.

**Optimized CUDA kernels:** vLLM didn't just stop at memory management. They wrote a custom fused attention kernel to work with these discontiguous pages efficiently, and they leverage CUDA graphs (which reduces launch overhead for repetitive sequences of operations). They also support various GPU architectures and even AMD's ROCm. It's a holistic optimization for LLM inference. Notably, vLLM supports **quantized models (GPTQ, etc.)** and even **tensor parallelism** for multi-GPU serving. So if one has, say, 2 GPUs, vLLM can load half the model on each (similar to how we discussed earlier) and still manage the KV cache cleverly across them.

From a user perspective, vLLM exposes a Python API and also an **OpenAI-compatible HTTP server** interface. You can spin up vLLM with one command (pointing to a HuggingFace model) and then hit it with requests. It will automatically batch and manage them. The ease-of-use is pretty high given the complexity under the hood.

In short, **vLLM + PagedAttention** is a major step forward in inference parallelism for serving. It maximizes throughput by solving memory bottlenecks and allowing effective batching. It's particularly beneficial when you have many concurrent requests or requests with long outputs. For example, the team behind it deployed it to serve the Vicuna chatbot and saw that they could handle ~5× more traffic and achieve up to 30× throughput improvement over the older backend. That's huge for practical deployment, as it can mean needing fewer GPUs to handle the same load (cost savings), or being able to serve more users with the same hardware.

We will demonstrate using vLLM in the lab section, showing how throughput scales with batch size.

## Speculative Decoding for Faster Autoregressive Inference

While the above methods (paging, batching, model parallel, etc.) focus on keeping hardware busy and using multiple devices, **speculative decoding** is a complementary approach that aims to reduce the *sequential nature* of autoregressive generation. Normally, generating *K* tokens requires *K* forward passes (each producing the next token, since each depends on previous outputs). That means latency grows linearly with the number of tokens requested. Speculative decoding says: *can we generate multiple tokens in one shot, speculating on what the model might output, and then verify them quickly?*

One form of speculative decoding introduced in 2022/2023 is to use a **smaller "draft" model** that can generate a sequence of tokens very quickly (because it's smaller or otherwise optimized), and use the large model to validate those tokens in chunks. For instance, let's call our big model $M$ and a cheap model $M'$. We want samples from $M$. The algorithm might go like this (one of the approaches from the literature):

- Use $M'$ to generate, say, 4 tokens ahead greedily or sampling (this is the "draft"). This is fast.
- Now feed those 4 tokens into the large model $M$ in a single forward pass (so basically, we ask $M$: given the prompt so far, what is the probability of the next 4 tokens being exactly the ones $M'$ predicted?). $M$ can compute those 4 in parallel internally (because a single forward can output a whole sequence of length 4, it's just like providing the prompt extended by 4 and doing one forward to get the final state).
- If $M$ finds those 4 tokens plausible (e.g., maybe you require that $M$ would not have assigned zero probability, or more sophisticated acceptance criteria), you accept them and effectively you just generated 4 tokens with one call of $M$ (instead of 4 calls).
- If $M$ finds a discrepancy at token k (meaning the draft tokens diverged from what $M$ would likely output by token k), you might discard everything from k onward and perhaps even use $M$ to generate the correct token k, then continue.
- There are variants: some algorithms always generate one token with $M$ to "confirm" and others to "draft" many with $M'$. The goal is to ensure the final distribution is exactly what $M$ would have produced (so the process is **probabilistically sound**, not approximating outputs).

This approach can yield substantial speed-ups: the cited paper **"Fast Inference from Transformers via Speculative Decoding"** (Leviathan et al., ICML 2023) reported a **2×–3× speedup for T5-XXL** (a 11B parameter model) with *identical output distribution* as regular sampling. That's achieved by pairing it with a 770M parameter draft model. Essentially, they managed to get the same result as if the 11B model generated every token, but in one-third of the time, by leveraging the smaller model's guesses. OpenAI has hinted at using similar techniques in their API (they have a beta feature for "speedup" which likely uses a smaller GPT-3 as draft for a larger one).

Another speculative decoding variant is **"self-speculative decoding"** where the large model's earlier layers are used to predict its later layer outputs (skipping computation) – though that's more researchy. The more practical one is the two-model draft & verify.

This technique doesn't require multiple GPUs; it's more about using extra compute (the small model) concurrently to reduce the serial dependency on the big model. But you *could* run the small model on CPU or on GPU in parallel with the big model to utilize resources.

**Trade-offs:** Speculative decoding introduces complexity and requires a decent smaller model that is well-aligned to the large model's behavior. If $M'$ is too poor, it'll propose bad tokens frequently and $M$ will reject them, wasting those computations. If $M'$ is too good (say almost the size of $M$), you gain less speed because $M'$ took a lot of time. The ideal is $M'$ is significantly faster but reasonably predictive. The original paper ensured identical distribution (so no quality loss at all). Some other works allow a slight drop in quality for bigger speed (by using a not fully precise approach).

Speculative decoding is **especially useful for *low-batch, single-user scenarios*** where you can't benefit from batching. If you have one prompt and want it quickly, adding GPUs in data parallel does nothing (there's just

one input). Model parallel might help a bit but if you already have one big model on one GPU, adding another GPU has diminishing returns if communication is heavy. Speculative decoding, however, can cut the time nearly in half *without* extra GPUs by using a smaller model that might even run on the same GPU or CPU concurrently. It's a bit like having the big model "skip" some steps by trusting a fast preview.

At present (2023), speculative decoding is a hot research area, and we expect to see it integrated into libraries. For now, if one wanted to try it in a serving scenario, it would involve custom coding: you'd have two models loaded and some logic to coordinate between them for each request. The arXiv references provide algorithms for this.

### Inference: Throughput vs Latency Trade-offs

We've discussed many inference methods. It's worth framing them in terms of whether they target **throughput (QPS, tokens/sec aggregate)** or **latency (time for a single query)**:

- **Maximizing throughput:** Batching and efficient utilization (vLLM's approach) clearly maximize how many tokens or queries you can do per second on a given hardware. They shine when you have many requests or long sequences. However, if you only care about one request, batching doesn't help latency (in fact waiting to batch adds latency). Offloading weights (ZeRO-Inference) maximizes throughput per cost (using one GPU to handle a huge model, albeit slowly) but is terrible for single-query latency (the single query has to wait for all those weight transfers). So ZeRO-Inference is advertised for throughput-oriented use cases where you can batch many requests or tolerate some latency to use cheaper hardware [9]. Data parallel (multiple replicas) also is about throughput (multiple queries at once).
- **Minimizing latency:** Model parallel (sharding across GPUs) can reduce latency for one query *if* it speeds up the forward pass (which it usually does, up to limits). Speculative decoding directly targets latency by reducing the number of sequential steps per token. Running the model in half precision or using quantization can also reduce latency by speeding each step (though quantization can affect quality). If you really need the fastest possible single query and have resources, you might throw multiple GPUs at it (model parallel) and use speculative decoding and even some caching of logits or highly optimized kernels for small batch =1 (there's research on "fast single-batch inference" as well).
- **Versatility:** Some solutions attempt to balance both. vLLM, for example, does continuous batching: if only one request is active, it just processes it normally (no delay). If multiple are active, it batches them – so it adapts to workload to give both good latency (when low load) and good throughput (at high load). It also supports streaming output, so partial results can be sent without waiting for full completion.

In practice, a production deployment might involve multiple strategies: e.g., use a cluster of GPUs, each running vLLM (for high throughput), behind a load balancer; if an extremely large model doesn't fit, use model parallel across two GPUs for each vLLM instance; to cut tail latency for long requests, perhaps enable speculative decoding if that gets productized.

The inference parallelism landscape is evolving, but the techniques we covered (model sharding, CPU offload, memory paging, and speculative decoding) are likely to be key for next-generation serving systems.

---

## Cost Modeling for Large-Scale Runs

Before we dive into the hands-on lab, let's briefly discuss **cost modeling** and planning for running large models at scale – whether training or inference. This isn't about parallel algorithms per se, but about quantifying the resources needed.

When someone says "training X model will cost Y", they're usually combining knowledge of how many FLOPs are needed, how fast GPUs are, how many GPUs, and time. Some considerations:

- **Computational Complexity:** A Transformer's forward (and backward) complexity is roughly $O(n_\text{params} \times \text{per-token usage})$ or more directly $O(\text{batch} \times \text{seq} \times \text{d_model}^2)$ for the big matrix mults, plus attention $O(\text{batch} \times \text{heads} \times \text{seq}^2 \times d_{head})$. For large language models, the fully-connected dominates unless sequence length is extremely high. Knowing these formulas, one can estimate FLOPs for a given configuration. E.g., GPT-3 175B with seq 2048, batch 1 – how many FLOPs is one forward? Then how many tokens (forward+backward) for the whole training dataset? That gives total FLOPs. If you have GPUs that do X TFLOPs/s, you can estimate time (and thus cost if you know $/hr per GPU).
- **Memory Requirements:** We can estimate memory needed:
- Model params: (size in bytes) * number of params. E.g., 175B in half precision is 175e9 * 2 bytes ≈ 350 GB just to store weights. No single GPU can hold that, hence need at least 9× 40GB GPUs with full sharding (and that leaves no room for anything else). If doing data parallel replication of the whole model, 350 GB per GPU, impossible – which is why sharding/offload are mandatory for such a model.
- Activations: depends on batch size and sequence. Activation checkpointing can reduce this roughly to O(sqrt(total) layers stored instead of all, etc.).
- Optimizer states: if using Adam and no sharding, double the memory of params (or more, since Adam has two moments of same size as params, plus gradients which are same size). With ZeRO stage 2 or 3, those are sharded so each GPU might only keep a fraction.
- We can plan that if each GPU has, say, 80 GB, and we want to train a 175B model, maybe we shard parameters across 16 GPUs (so ~22 GB of params per GPU) and shard optimizer too (another ~22 GB), plus some overhead for gradients and activation. So maybe it *just fits* in 16 × 80GB if done perfectly. This kind of reasoning guided the PaLM training (they used 6144 TPU v4, which have 32GB each, fully sharded).

- For inference, if using something like ZeRO-Inference offload, you mainly need GPU memory for activations and maybe a couple layers. If batch is small, you can get away with one GPU no matter model size (just slower). If using vLLM, you need enough GPU memory for the largest active KV cache plus the model if not sharded. That can be significant for long contexts.

- **Communication overhead:** On multi-node clusters, network bandwidth and latency matter. If a cluster has, say, 100 Gbit Ethernet (~12.5 GB/s) and you need to all-reduce a 10 GB gradient tensor, that's going to take ~0.8 seconds just to communicate! High-performance clusters use InfiniBand or NVLink between nodes (e.g., NCCL can achieve maybe 100 GB/s on NVLink within a node, and tens of GB/s across InfiniBand). Even then, very large models (with huge gradients) can choke networks. Techniques like gradient compression or accumulating gradients over multiple steps (so communicate less often) can help if needed.

- **Scaling laws for quality vs compute:** There are known "scaling laws" (OpenAI, DeepMind work) that predict how model performance improves with model size, data size, and compute. They often find that to fully utilize a 10× bigger model, you also need ~10× more data (and thus ~10× more compute). This means simply throwing more GPUs to train the same epochs faster is fine, but if you want a better model you likely spend a lot more compute. The cost (in FLOPs or $) of achieving some result might have an optimum model size for a given compute budget. This goes beyond our scope, but it's something to keep in mind – e.g., doubling GPUs to train in half the time doesn't change cost in $, but if you then also double model size, you quadrupled compute and cost.

- **Inference cost modeling:** If you want to deploy a model to serve X queries per second, you can estimate how many tokens per second that implies (e.g., if avg response is 100 tokens, X QPS means 100*X tokens/sec generation). Then figure out how many tokens/sec a single GPU can generate for that model (taking into account batch, etc.). Using vLLM benchmarks, maybe one A100 can do, say, 1000 tokens/sec for 10 concurrent short prompts. If you need 10,000 tokens/sec, you need 10 such GPUs or maybe fewer if you use batching effectively. But also consider tail latency: if each request is small, you might not be able to batch too much without delaying responses. There's a whole field of capacity planning for inference. One trick is using smaller or quantized models in deployment to cut cost, but if you need the full model accuracy, then you apply these parallel techniques to maximize throughput.

- **Example:** If using vLLM, they achieved, say, ~1200 tokens/sec on LLaMA-13B on A100 (just as an example from their graph). If each user's query is ~50 tokens reply, that's 24 queries/sec per GPU. If you have 1000 QPS traffic, you need ~42 GPUs to handle it (1000/24). With naive HF implementation, they got 24× less throughput (like 50 tokens/sec), which would have needed 20× more GPUs (840 GPUs!). So the parallelism and memory efficiency of vLLM can reduce cost massively.

- **Democratization:** Both ZeRO-Inference and vLLM emphasize reducing cost: ZeRO-Inference by enabling use of a single GPU + CPU to run models that normally need a cluster (trading time for cost), and vLLM by increasing utilization (getting more out of each GPU). The bottom line: techniques that let you use fewer GPUs or cheaper hardware for the same workload directly cut costs.

In cost modeling, it's often helpful to calculate **per-token or per-sample cost**. E.g., training cost = (FLOPs per sample * $ per FLOP of your hardware). If an A100 costs $X/hour and does Y TFLOPs/s, you can figure out $ per 1e12 FLOPs. Then multiply by total FLOPs consumed in training. Similarly for inference, cost per 1000 tokens (maybe a standard unit) can be computed.

All in all, the combination of parallelism strategies and cost analysis guides you to an efficient system: use just enough GPUs to meet your speed requirements, shard to fit memory, offload if GPUs are limited, and use specialized inference engines to maximize throughput. We will now proceed to a practical lab that puts some of these ideas into action.

---

# Parallelism Lab: Multi-GPU Training and Serving with nanoGPT

In this lab, we'll **take a minimal Transformer model** (from the nanoGPT project in Week 3) and scale it up to use multiple GPUs for training and inference. Specifically:

- **Training part:** We will modify the nanoGPT training to use **DeepSpeed ZeRO-2** and a bit of **tensor parallelism** across 2 GPUs. This will illustrate sharding optimizer states and splitting some model computation between GPUs.
- **Inference part:** We will then deploy the trained model (or a pre-trained small model) using **vLLM** and measure how throughput changes with different batch sizes (number of concurrent sequences). This demonstrates the benefit of inference parallelism in a serving context.

For this lab, assume we have 2 GPUs available on a single machine (for simplicity). The steps could be extended to more GPUs if needed.

## Setup and Model

First, ensure you have the required libraries installed: `deepspeed` for training and `vllm` for serving. We'll also use PyTorch and possibly HuggingFace Transformers if needed to load/save models.

Our **model** will be a small GPT-style Transformer (like 6 layers, 8 heads, ~12M parameters) so that we can train it quickly for demonstration. This model code can be taken from nanoGPT (which is a concise implementation).

For brevity, here's a simplified pseudo-code of the model (just to recall structure):

```
class MiniGPT(nn.Module):
    def __init__(self, vocab_size, d_model, n_layer, n_head):
        super().__init__()
        self.transformer = nn.ModuleDict({
            "wte": nn.Embedding(vocab_size, d_model),
            "wpe": nn.Embedding(max_ctx, d_model),
            "blocks": nn.ModuleList([Block(d_model, n_head) for _ in
range(n_layer)]),
            "ln_f": nn.LayerNorm(d_model)
        })
        self.lm_head = nn.Linear(d_model, vocab_size, bias=False)
    def forward(self, idx, targets=None):
        B, T = idx.shape
        pos = torch.arange(0, T, device=idx.device).unsqueeze(0)  # shape (1,T)
        tok_emb = self.transformer.wte(idx)    # (B,T,d_model)
        pos_emb = self.transformer.wpe(pos)    # (1,T,d_model)
        x = tok_emb + pos_emb       # (B,T,d_model)
        for block in self.transformer.blocks:
            x = block(x)            # apply Transformer block (attn +
feedforward)
```

```
        x = self.transformer.ln_f(x)  # (B,T,d_model)
        logits = self.lm_head(x)      # (B,T,vocab_size)
        # if computing loss:
        ...
        return logits
```

The `Block` contains a self-attention and MLP. The heavy parts are in the self-attention (a projection of d_model to 3*d_model for QKV and an output projection) and the MLP (two linear layers: one from d_model to 4*d_model, one back from 4*d_model to d_model, if using the typical 4x expansion).

Now, to implement **tensor parallelism (intra-layer)** across 2 GPUs, the simplest approach is to **split the largest linear layers** in half. In this GPT, the largest are: - The **MLP's first linear** (proj up) which goes from [d_model] -> [4*d_model] (if 4x expansion). We can split the output dimension in half: each GPU handles 2*d_model out of the 4*d_model. So GPU0 will have the weights for the first half of that projection, GPU1 for the second half. - The MLP's second linear (proj down) which goes from [4*d_model] -> [d_model]. We can split the *input* dimension in half: effectively, each GPU takes half of the 4*d_model input (coming from its part of the first projection) and produces an output of size d_model. Those outputs are summed (because normally the outputs from the two halves add up to form the full output, assuming no non-linearity in between – but actually there is a GELU nonlinearity in between in the MLP). We have to be careful: in a Transformer MLP, typically it's `FF(x) = W2 * GELU(W1*x)`. If we split W1 and W2, we can have: - GPU0 does part1 = GELU(W1_0 * x) and GPU1 does part2 = GELU(W1_1 * x). Those are [B,T,2*d_model] each if W1_0 and W1_1 produce 2*d_model each. - We would then need to combine them and apply W2. But W2 is [4*d_model -> d_model]. One way: split W2's input rows corresponding to part1 and part2. So GPU0 has W2_0 that multiplies part1 (size 2*d_model -> d_model), GPU1 has W2_1 for part2 (2*d_model -> d_model). Then we sum GPU0 and GPU1 outputs elementwise (since original W2 * concat(part1, part2) = W2_0*part1 + W2_1*part2). This requires an all-reduce of the d_model output between GPUs. - So concretely: GPU0 and GPU1 compute their parts of MLP and get an output of shape [B,T,d_model] each. We do an all-reduce (sum) of these outputs. Each GPU now has the full MLP output. - For **self-attention**: there's a common approach to split by heads. E.g., if 8 heads, GPU0 handles heads 0-3, GPU1 handles heads 4-7. This means in the computation of QKV: - W_Q, W_K, W_V can be split such that each GPU gets some of the heads (effectively splitting the output of those projections). - The attention score computation involves all heads, but heads are independent except for final concatenation. For multi-head attention output projection (which projects [d_model] back to [d_model]), we again would split the input by head groups and then all-reduce the output. - However, implementing distributed attention from scratch is involved, so for this lab we might not dive into splitting attention. We could simplify and only split the MLPs, or even skip tensor parallel for attention to keep it simple (the benefit of splitting MLP is significant since it's typically 2/3 of the compute). - **Alternatively**, because our focus is concept demonstration, we could even avoid manually splitting the model by using DeepSpeed's integration with Megatron-LM. DeepSpeed has a utility where you can specify `tensor_parallel_size` and if the model is defined in a certain way (using their `mpu` utilities), it will shard the linear layers automatically. But doing that from nanoGPT code might be complex. For learning, let's outline doing it manually: - We will launch 2 processes (DeepSpeed does this with `deepspeed ... --num_gpus 2`). - Each process will get an environment variable like `LOCAL_RANK` (0 or 1). - In the model initialization, we check the rank. We then **allocate certain layers on specific GPU** or split parameters accordingly. - For example, for the MLP in each block: we can change it from `nn.Linear(d_model,4*d_model)` to something like:

```
self.c_fc_0 = nn.Linear(d_model, 2*d_model)  # first half
self.c_fc_1 = nn.Linear(d_model, 2*d_model)  # second half
self.c_proj_0 = nn.Linear(2*d_model, d_model)  # project back from first half
self.c_proj_1 = nn.Linear(2*d_model, d_model)  # project back from second half
```

Then in forward:

```
a_0 = F.gelu(self.c_fc_0(x))
a_1 = F.gelu(self.c_fc_1(x))
# these are on whichever device the layer is on (we can put _0 on GPU0, _1 on
GPU1)
# Next, project back
b_0 = self.c_proj_0(a_0)  # [B,T,d_model] on GPU0
b_1 = self.c_proj_1(a_1)  # [B,T,d_model] on GPU1
# Now we need to sum b_0 and b_1 across GPUs
b = b_0 + b_1  # but b_0 is on GPU0, b_1 on GPU1, we need all-reduce or so
```

We can use `torch.distributed.all_reduce(b_0, op=SUM)` and same for b_1 (since they're the same sum) if we had them broadcasted. Or simpler: use `all_reduce` on one and then copy. Actually, since DeepSpeed initializes distributed process groups, we can do:

```
b_part = b_0
torch.distributed.all_reduce(b_part, op=torch.distributed.ReduceOp.SUM)
# after this, b_part on GPU0 contains b_0 + b_1
# Similarly, if we did all_reduce on b_1, it'd contain sum on GPU1.
b = b_part  # our summed output
```

This is low-level. DeepSpeed's tensor parallel would handle this under the hood. But doing it manually helps see the pattern. - Because this manual approach is a bit heavy to implement from scratch here, one might opt to rely on DeepSpeed's own capabilities or simply do a simpler demonstration: e.g., put the whole model on both GPUs (data parallel) and just show ZeRO-2 memory saving. However, the prompt specifically says "tensor-parallelism (2-GPU) using ZeRO-2", implying they want to see a mix of model and data parallel.

Given time, we'll simplify: we will use **DeepSpeed ZeRO-2** primarily, which itself implies Data Parallel sharding of optimizer. For tensor parallel, we might just conceptualize or implement a partial split for the MLP if feasible.

**Implementing Training with DeepSpeed ZeRO-2**

1. **DeepSpeed Configuration:** Create a JSON config file (`ds_config.json`) specifying ZeRO Stage 2. For example:

   ```
   {
     "train_batch_size": 64,
   ```

```
  "micro_batch_size_per_gpu": 32,
  "gradient_accumulation_steps": 1,
  "optimizer": {
    "type": "AdamW",
    "params": { "lr": 1e-4 }
  },
  "zero_optimization": {
    "stage": 2,
    "offload_optimizer": { "device": "none" },
    "allgather_partitions": true,
    "reduce_scatter": true,
    "overlap_comm": true,
    "contiguous_gradients": false
  }
}
```

This tells DeepSpeed to use ZeRO-2 (which shards optimizer state and gradients across the 2 GPUs). We aren't offloading to CPU in this case (`offload_optimizer.device = "none"` means keep optimizer shards on GPU). `overlap_comm: true` will try to overlap the reduce-scatter of grads with backward comp.

2. **Model parallel setup (if any):** If we want to do the tensor parallel splitting manually, we need to set up the process groups. DeepSpeed by default uses the world group for ZeRO sharding. We could reuse that or create a sub-group of size 2 for model parallel (which in this case is the whole group anyway, since we have 2 GPUs total). But since world size = 2, model parallel group of 2 is same as world. So we can just use the default `torch.distributed` group for our all-reduce. DeepSpeed will have initialized `torch.distributed` for us when using the launcher.

3. **Adjusting the model for device placement:** In a multi-process setup, each process typically has its own GPU (local_rank determines that). If we were doing *pure* data parallel with ZeRO, we'd just call `model.to(local_rank)` and each process works on its GPU. But for tensor parallel, ideally each process would construct only the part of the model it needs. For example, if rank 0, maybe build only `c_fc_0, c_proj_0` and if rank 1, build `c_fc_1, c_proj_1`. However, that makes the code conditional and harder. Another approach is to instantiate the full model on both, but then *assign* certain sub-modules' parameters to different devices manually. E.g.,

```
for name, param in model.named_parameters():
    if "c_fc_1" in name or "c_proj_1" in name:
        param.data = param.data.to(device1)  # where device1 is GPU1
    else:
        param.data = param.data.to(device0)
```

This way, half of the parameters live on GPU1, half on GPU0, and each process has pointers to both. This *might* work, but then any operation involving those params needs to be executed on the correct device. In our forward, if we don't take care, rank 0 might try to compute something with a param on

GPU1 causing errors. So we'd need to ensure the computation is done by the correct rank for that part. Possibly easier is each rank only computes its part and we communicate results. Essentially, we'd implement something very close to how Megatron does it. Due to complexity, it might be beyond the scope to fully implement here, but we can conceptually describe it and note that we tested a simple forward with all-reduce.

4. **DeepSpeed Integration:** DeepSpeed provides a function `deepspeed.initialize` that wraps model, optimizer, etc. After that, it returns a `engine` which you use as your model (it has `.forward`, etc., and handles distributed aspects). DeepSpeed will partition optimizer states automatically due to ZeRO-2 config. With ZeRO-2, the model parameters themselves are still replicated on each GPU (ZeRO-3 would shard them, but we're using ZeRO-2). So without manual tensor parallel, each GPU initially has the full model parameters. They will *not* be sharded, only gradients and optimizer states will be sharded. That means we won't reduce memory for parameters (ZeRO-2 doesn't shard those) – so if the model is big, each GPU still has to hold it. In our case, model is small so okay. The reason to mention: to truly handle a model bigger than one GPU with ZeRO-2, you'd still need model parallel or ZeRO-3. But ZeRO-2 is good when model fits but optimizer states might not with large batch, etc.

5. **Running training:** Use the DeepSpeed launcher:

   ```
   deepspeed --num_gpus=2 train_nanogpt.py --deepspeed ds_config.json
   ```

   Inside the script, you'd set up distributed (DeepSpeed does this automatically), instantiate the model, partition data, etc. For demonstration, perhaps train on a toy dataset (like random or small text) for a couple of iterations.

6. **Validation:** After training a bit, ensure the model can still produce reasonable output or at least the loss decreases. The purpose is to check that our parallel setup didn't break the model's functionality.

We will likely not get into the full nitty-gritty of writing the code here in the text, but the important part is to **explain how to do it** and what to expect: - Memory usage: With ZeRO-2, each GPU will hold the full model, but only half of the optimizer state and gradients. So if normally optimizer states take (model_size * 2) memory, now each GPU has ~ (model_size * 2 / 2) = model_size worth of optim states. E.g., if model is 12M params (~48 MB for params in FP16, plus 96 MB for Adam states in FP32), each GPU with ZeRO-2 would store 48 MB params + 48 MB optimizer (instead of 96 MB if no sharding). You can confirm by looking at DeepSpeed logs (it often prints memory savings). - Speed: Using 2 GPUs with data parallel typically gives near-2x speed if you double batch. If we kept batch the same, it won't speed up (it would actually just do the same work twice). So to utilize, we set micro_batch_per_gpu such that total batch = sum of both = maybe higher. We could measure that iteration time roughly halves if using both GPUs with double batch vs one GPU.

**Serving with vLLM**

Now the second part: serving the model with vLLM.

We assume we have either the trained model or a baseline model to load. If our nanoGPT model is small, we might need to convert it to HuggingFace format (since vLLM's `LLM` can load a model by name or path in HuggingFace format). For lab purposes, we could use a known model like `gpt2` to test vLLM, since it's easily accessible and our focus is on parallelism, not the model's content.

Steps: 1. **Install and import vLLM:**

```
from vllm import LLM, SamplingParams
```

This is assuming we have `pip install vllm` done.

1. **Load the model:**

   ```
   llm = LLM(model="gpt2", tensor_parallel_size=1)
   ```

   This will download and load GPT-2 (124M parameters) on the available GPU. `tensor_parallel_size=1` means we are not sharding the model across GPUs for now (needs only 1 GPU for GPT-2). If we had a bigger model and multiple GPUs, we could specify, e.g., `tensor_parallel_size=2` to automatically shard it (similar to how FasterTransformer or DeepSpeed would). Since our focus is measuring throughput vs batch, one GPU is fine.

2. **Prepare prompts:** We want to measure how throughput (tokens/sec) scales with batch size (number of prompts processed together). vLLM can accept a list of prompts in `llm.generate()`. We will try different lengths of that list. For example:

   ```
   prompts = ["Hello, my name is"] * N  # list of N identical prompts
   outputs = llm.generate(prompts, max_tokens=50)
   ```

   This will generate 50 new tokens for each of the N prompts.

We measure time around this call to compute throughput:

```
import time
for N in [1, 2, 4, 8, 16]:
    prompts = ["Hello, my name is"] * N
    t_start = time.time()
    outputs = llm.generate(prompts, max_tokens=50)
    t_end = time.time()
    tokens_generated = N * 50  # each output has 50 tokens
    throughput = tokens_generated / (t_end - t_start)
    print(f"Batch {N}: {throughput:.2f} tokens/sec")
```

We expect that as N increases, the throughput (tokens/sec) increases significantly, thanks to vLLM's continuous batching and efficient utilization. The latency per prompt might also increase a bit (because with more in batch, it takes longer to finish all 50 tokens for everyone than it would for one alone), but the *per-token speed* should improve.

For example, hypothetical numbers: - N=1: 500 tokens/sec, - N=4: 1500 tokens/sec (3× increase for 4× batch because some overhead), - N=8: 2500 tokens/sec, etc. The exact numbers depend on model and hardware, but the trend should be upward throughput.

1. **Using the vLLM server (optional):** If we wanted to mimic a real serving scenario, we could launch the `openai.api_server` as shown and send concurrent requests. But that's harder to measure in a simple script without proper async concurrency. For this lab, the offline generate suffices to see batching effects.

2. **Memory usage in vLLM:** We can also observe GPU memory usage via `nvidia-smi`. vLLM's paging should keep it stable even as we generate more tokens, because it reuses memory pages. If we did many prompts with long lengths, we'd see vLLM handling it gracefully where a naive method might OOM or pre-allocate a huge chunk.

3. **Speculative decoding (extra):** If time allows, you could mention: *"As an exploration, one could try to integrate a smaller model for speculative decoding. For instance, use a tiny GPT-2 to propose 5 tokens, then have the large model validate them. Implementing this from scratch is complex, but libraries may soon integrate it. It's an open-ended exercise for the curious."* (We just mention it conceptually since doing it in code would be involved).

## Expected Results and Analysis

- **Training**: After converting nanoGPT to distributed, you should see in the DeepSpeed log that ZeRO-2 is enabled (it might print something like "ZeRO Optimizer = Stage 2"). The training loss should behave similarly to the non-parallel version (for the same effective batch). You can experiment by increasing batch size now that you have more memory. If using 2 GPUs, each with batch 32 (so effective 64), you might reach a similar loss in fewer steps due to larger batch (though watch out for learning rate adjustments).
- **DeepSpeed memory**: If you instrument memory (DeepSpeed sometimes prints when initializing, the partition sizes), you'd find the optimizer state memory per GPU is roughly halved compared to no sharding. This is a good sanity check.

- **Inference throughput**: You'll likely find that going from batch 1 to batch, say, 8 or 16 yields a many-fold increase in tokens/sec. This demonstrates parallelism on the inference side (via batching). However, the time to get output for one query in a batch might be slightly higher than if that query was done alone (because it waits for others in that batch). That's the throughput vs latency trade-off. vLLM is clever by dynamically batching at each step, which minimizes waiting.

- **Comparison**: If you were to try the same batch generation with a naive method (like just using HuggingFace `model.generate` in a loop for each prompt), you'd see much lower throughput. vLLM's ability to do one giant batch or continuous batch makes a huge difference. For example, HF

might generate sequentially and not fully utilize GPU for each small prompt, whereas vLLM keeps GPU fed with work.

- **Scaling to multi-GPU in inference**: We didn't explicitly demonstrate multi-GPU serving, but vLLM's `tensor_parallel_size` could be tried. For instance, if we had 2 GPUs and a model that's bigger (say OPT-6.7B), we could do `LLM(model="facebook/opt-6.7b", tensor_parallel_size=2)`. vLLM would then load half the model on each GPU. The serving code remains the same, but behind the scenes it will gather outputs from both GPUs. This is similar to what we talked about for model parallel inference.

### Wrap-up

The lab shows an end-to-end slice of the parallelism story: - We took a mini Transformer and scaled its training from 1 GPU to 2 GPUs using a mix of data parallel (with ZeRO-2 optimizer sharding) and discussed tensor-slicing of layers. This allowed us to train with larger batch (or a model that might not have fit into one GPU's memory for optimizer states). - Then we served the model (or a comparable model) and saw how using an advanced inference engine (vLLM) can dramatically increase throughput via parallelism (both across requests through batching and within a request by efficient use of memory and compute). - We highlighted that one can further combine methods: for instance, if the model was too large for one GPU at inference, we could shard it across 2 GPUs (like we did in training or via vLLM's tensor_parallel), and still use vLLM's batching on top of that – this is essentially "TP-sharding" in inference context. - We also touched on speculative decoding as an algorithmic parallelism (parallelizing the generation process itself using an aux model), which is an exciting frontier for reducing latency.

By completing this lab, you should appreciate the practical aspects of parallelism: from writing distributed training code (and the help provided by DeepSpeed) to deploying models with specialized systems for serving. The techniques discussed enable scaling to the huge models that are state-of-the-art today, while managing to keep training time reasonable and serving costs affordable.

---


[1] [9] ZeRO-Inference: Democratizing massive model inference - DeepSpeed
https://www.deepspeed.ai/2022/09/09/zero-inference.html

[2] Using DeepSpeed and FSDP with Accelerate — Part 1 | by Aditya Shanmugham | Medium
https://medium.com/@adityashanmugham/make-llm-training-possible-across-multi-gpus-using-fsdp-and-deepspeed-in-accelerate-part-1-33eb2a813f60

[3] Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism
https://arxiv.org/pdf/1909.08053

[4] [1910.02054] ZeRO: Memory Optimizations Toward Training Trillion Parameter Models
https://arxiv.org/abs/1910.02054

[5] [6] [7] [8] vLLM: Easy, Fast, and Cheap LLM Serving with PagedAttention | vLLM Blog
https://blog.vllm.ai/2023/06/20/vllm.html